

- [Rapport — TP Kubernetes / Service Mesh — LaboTrack](#)
  - [1. Contexte et environnement](#)
  - [2. Étape 1A — Manipulations Kubernetes \(20 questions\)](#)
  - [3. Étape 1B — Spring Boot demo \( monservice \)](#)
    - [3.1 Cas 1 — Dockerfile mono-stage + docker-compose](#)
    - [3.2 Cas 2 — Dockerfile multi-stage](#)
    - [3.3 Quel est l'intérêt de la technique multi-stage ?](#)
  - [4. Étape 2 — LaboTrack](#)
    - [4.1 Architecture](#)
    - [4.2 Cycle de vie type d'un échantillon](#)
    - [4.3 Multi-stage Dockerfiles](#)
    - [4.4 Build & push images vers la registry](#)
    - [4.5 Manifests Kubernetes](#)
    - [4.6 Service Mesh Linkerd](#)
    - [4.7 ServiceProfile — retries & timeouts](#)
    - [4.8 ServerAuthorization — zero-trust](#)
    - [4.9 RunBook](#)
    - [4.10 Déroulement de la démo](#)
  - [5. Limites et pistes d'amélioration](#)
  - [6. Annexes](#)

# Rapport — TP Kubernetes / Service Mesh — LaboTrack

Module Architecture — INSA — H. Tondeur 2026

Équipe : - Mikael LE CAM - Antonin RIQUART - Jonathan ISAMBOURG - Gregoire LEGRAND

## 1. Contexte et environnement

| Outil                | Version             |
|----------------------|---------------------|
| Ubuntu               | 24.04.4 LTS (Noble) |
| Docker Engine        | 29.4.2              |
| Docker Compose       | v5.1.3              |
| OpenJDK              | 21.0.10             |
| Maven                | 3.8.7               |
| kubectI              | v1.36.0             |
| Minikube             | v1.38.1             |
| Kubernetes (cluster) | v1.35.1             |
| Linkerd CLI          | edge-26.5.1         |

Le fichier `bootstrap-wsl.sh` à la racine du dépôt automatise toute cette installation pour reproduire l'environnement.

## 2. Étape 1A — Manipulations Kubernetes (20 questions)

Réponses détaillées (commande + capture pour chaque question) dans `etape1/questions/answers.md`.

Captures dans `etape1/questions/screenshots/Q1.png` à `Q21.png`.

## 3. Étape 1B — Spring Boot demo ( monservice )

Service REST minimal en Spring Boot 3.3.5 / Java 21, exposant :

- GET `/monservice/echo/{nom}` → `{"message":"echo: {nom}"}`
- POST `/monservice/hello` (body JSON `{"nom":"..."}`) → `{"message":"Hello {nom}"}`

Détails dans `etape1/monservice/README.md` ; captures pas-à-pas dans `etape1/monservice/screenshots/c1.png` à `c11.png`.

### 3.1 Cas 1 — Dockerfile mono-stage + docker-compose

**Pré-requis** : la fat-jar doit avoir été produite localement (`mvn clean package`).

FROM eclipse-temurin:21-jre

```

WORKDIR /app
COPY target/monservice.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]

```

```

services:
  monservice:
    build: { context: ., dockerfile: Dockerfile.simple }
    image: monservice:simple
    ports: ["8080:8080"]

```

Test en local (preuve de bon fonctionnement non conteneurisé) puis depuis le conteneur (preuve d'accès au container en cours d'exécution — détails et captures dans `etape1/monservice/README.md` .

### 3.2 Cas 2 — Dockerfile multi-stage

```

FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/monservice.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]

```

Build, run et tests en une passe, sans `mvn` préalable côté hôte. Captures `c8.png` à `c10.png` .

### 3.3 Quel est l'intérêt de la technique multi-stage ?

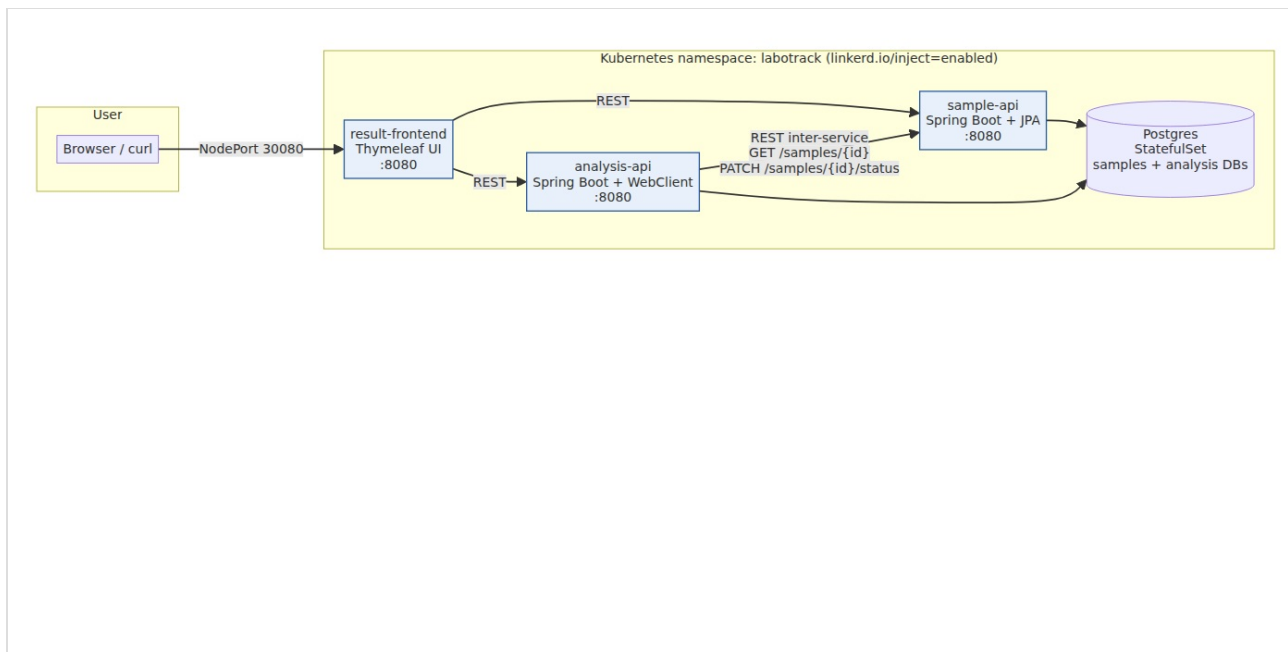
| Critère                     | Mono-stage (cas 1)                                                                         | Multi-stage (cas 2)                            |
|-----------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------|
| Taille image finale         | JRE + jar (~330 Mo)                                                                        | JRE + jar (~330 Mo, identique car même base)   |
| Outils dans l'image         | JRE + fat-jar                                                                              | JRE + fat-jar                                  |
| Dépendance hôte             | JDK + Maven obligatoires ( <code>mvn package</code> avant <code>docker compose up</code> ) | aucune (Docker suffit)                         |
| Étapes côté CI              | <code>mvn package</code> + <code>docker build</code>                                       | <code>docker build</code> (une seule commande) |
| Reproductibilité            | dépend de la version Java/Maven hôte                                                       | versions épinglées dans la phase de build      |
| Cohérence prod ↔ CI ↔ local | risque (fat-jar produit sous JDK différent)                                                | garantie (build dans l'image)                  |

**Note** : ici, le bénéfice de **taille** est nul car les deux Dockerfiles utilisent la même base `eclipse-temurin:21-jre` . Le gain de taille apparaît seulement face à une image naïve qui laisserait Maven et le JDK dans la couche finale (~700 Mo). Le vrai bénéfice du multi-stage dans notre cas est l'**indépendance de l'hôte** et la **reproductibilité**.

## 4. Étape 2 — LaboTrack

### 4.1 Architecture

3 microservices ( `sample-api` , `analysis-api` , `result-frontend` ) + une base PostgreSQL partagée (deux schémas), tous déployés dans le namespace `labotrack` annoté `linkerd.io/inject=enabled` pour bénéficier automatiquement du sidecar Linkerd.



Architecture LaboTrack

Description détaillée et diagramme source mermaid : `architecture.md`.

## 4.2 Cycle de vie type d'un échantillon

1. **Création** — result-frontend appelle POST /samples sur sample-api (statut REGISTERED).
2. **Analyse** — result-frontend appelle POST /analyze/{id} sur analysis-api.
3. **Inter-service** — analysis-api appelle GET /samples/{id} sur sample-api, génère un résultat aléatoire (glycémie  $\in [0.65 ; 1 \text{ g/L}]$ ), persiste, puis met à jour le statut à VALIDATED via PATCH /samples/{id}/status.
4. **Restitution** — result-frontend interroge à la fois sample-api et analysis-api pour afficher le tableau agrégé.

## 4.3 Multi-stage Dockerfiles

Les trois services partagent un Dockerfile multi-stage identique au Cas 2 ci-dessus (cf. § 3.2). Une seule commande `docker build` produisant l'image OCI prête au déploiement, sans dépendance hôte.

## 4.4 Build & push images vers la registry

Les 3 images sont buildées puis poussées dans la registry interne de Minikube.

### 4.4.1 Activation de la registry Minikube

```
minikube addons enable registry
```

```

mikaël@DESKTOP-LN3JADJ: /mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ minikube addons enable registry
Registry addon with docker driver uses port 60952 please use that instead of default port 5000

For more information see: https://minikube.sigs.k8s.io/docs/drivers/docker
  • Using image registry.k8s.io/minikube/kube-registry-proxy:v0.0.11
  • Using image docker.io/registry:3.0.0
Verifying registry addon...
The 'registry' addon is enabled
mikaël@DESKTOP-LN3JADJ: /mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ kubectl -n kube-system get pods -l kubernetes.io/minikube-addons=registry
NAME                                READY    STATUS    RESTARTS   AGE
registry-7984dc5f99-sgp26           1/1     Running   0           14s
registry-proxy-mz5q7                1/1     Running   0           14s

```

Registry addon activée

Le pod `registry` tourne dans `kube-system`. On l'expose sur `localhost:5000` côté hôte via un `kubectl port-forward` :

```
kubectl -n kube-system port-forward --address 127.0.0.1 svc/registry 5000:80 &
curl -s http://localhost:5000/v2/_catalog # -> {"repositories":[]}
```

```

mikaël@DESKTOP-LN3JADJ: /mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ kubectl -n kube-system port-forward svc/registry 5000:80 &
PF_REG=$!
sleep 2
curl -s http://localhost:5000/v2/_catalog
[1] 21870
Forwarding from 127.0.0.1:5000 -> 5000
Forwarding from [::]:5000 -> 5000
Handling connection for 5000
{"repositories":[]}
```

Catalog initial vide

## 4.4.2 Build des images sur le docker hôte

On bascule explicitement sur le docker WSL (et non celui de minikube) :

```
eval $(minikube docker-env -u)
docker info --format 'Server = {{.ServerVersion}}'
```

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ eval $(minikube docker-env -u) 2>/dev/null || true
docker info --format 'Context = {{.OperatingSystem}} | Server = {{.ServerVersion}}'
Context = Docker Desktop | Server = 29.3.1
```

Docker hôte sélectionné

Build de sample-api (multi-stage, une seule passe) :

```
docker build -t localhost:5000/sample-api:1.0 labotrack/services/sample-api
```

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ docker build -t localhost:5000/sample-api:1.0 services/sample-api
[*] Building 59.3s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 3kB
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre
=> [internal] load .dockerignore
=> => transferring context: 67B
=> [build 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:8e7dc4215c70f922e798c9f8aafa8a3734ca386342027b3dbb17cecc4a429c8e
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:8e7dc4215c70f922e798c9f8aafa8a3734ca386342027b3dbb17cecc4a429c8e
=> [internal] load build context
=> => transferring context: 8.03kB
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre@sha256:ff65ff0d43c73d2b675eb4b758665a5cb487e7df127436a9979f8172c144c819
=> => resolve docker.io/library/eclipse-temurin:21-jre@sha256:ff65ff0d43c73d2b675eb4b758665a5cb487e7df127436a9979f8172c144c819
=> CACHED [stage-1 2/2] WORKDIR /app
=> CACHED [build 2/6] WORKDIR /src
=> [build 3/6] COPY pom.xml .
=> [build 4/6] RUN mvn -B -q dependency:go-offline
=> [build 5/6] COPY src ./src
=> [build 6/6] RUN mvn -B -q -DskipTests package
=> [stage-1 3/3] COPY --from=build /src/target/sample-api.jar app.jar
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:73859d098a4f7287f441510936cf71a3695edu242f71698f62c4028b47d04b6
=> => exporting config sha256:e3d2928cf5f4bdcdb9704183407c976de78afa5301142dd412406f587fa8a6e
=> => exporting attestation manifest sha256:3d1bfe355e2468ee1e24071c089af21aa5dc3f0ae94f6d894df8efa7aa97e34
=> => exporting manifest list sha256:4b25fbdecdf68b9d78c44aab7a26f67ac49c71b1a4d3820986a9b9935be665d
=> => naming to localhost:5000/sample-api:1.0
=> => unpacking to localhost:5000/sample-api:1.0
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ docker images | grep localhost:5000
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
localhost:5000/sample-api:1.0 4b25fbdecdf6 503MB 147MB
```

docker build sample-api

Configuration du daemon Docker pour reconnaître la registry HTTP :

```
sudo tee /etc/docker/daemon.json >/dev/null <<'EOF'
{ "insecure-registries": ["localhost:5000", "127.0.0.1:5000"] }
EOF
sudo systemctl restart docker
docker info | grep -A3 'Insecure Registries'
```

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ sudo mkdir -p /etc/docker
echo '{"insecure-registries":["localhost:5000","127.0.0.1:5000"]}' | sudo tee /etc/docker/daemon.json
{"insecure-registries":["localhost:5000","127.0.0.1:5000"]}
```

Insecure registries

**Note technique** : malgré la configuration insecure-registries , le daemon Docker dans cet environnement WSL2 a continué à tente HTTPS avec i/o timeout. On bascule sur skopeo qui accepte --dest-tls-verify=false par invocation et bypassse complètement la config du daemon. Le résultat dans la registry est strictement identique : même format OCI, mêmes blobs, même catalogue.

## 4.4.3 Push avec skopeo

```
sudo apt-get install -y skopeo
skopeo copy --dest-tls-verify=false \
docker-daemon:localhost:5000/sample-api:1.0 \
docker://127.0.0.1:5000/sample-api:1.0
```

```

mikaël@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ skopeo copy --dest-tls-verify=false \
  docker-daemon:localhost:5000/sample-api:1.0 \
  docker://127.0.0.1:5000/sample-api:1.0
Getting image source signatures
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Copying blob fc9116e853d5 done
Copying blob 845dac72d2b2 done
Copying blob 2cce48cf6a50 done
Copying blob 9a376521bf1b done
Copying blob e38940369fdf done
Copying blob 538812a4b9bd done
Copying blob f3905ff52117 done
Copying config e3d2920cf5 done
Writing manifest to image destination
mikaël@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$

```

skopeo copy / push

Vérification immédiate côté registry :

```

curl -s http://127.0.0.1:5000/v2/_catalog
# -> {"repositories":["sample-api"]}

```

```

mikaël@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ curl -s http://127.0.0.1:5000/v2/_catalog
curl -s http://127.0.0.1:5000/v2/sample-api/tags/list
{"repositories":["sample-api"]}
Handling connection for 5000
{"name":"sample-api","tags":["1.0"]}

```

Catalog après le 1er push

Push des deux autres services (les images ont été exportées depuis le docker de Minikube via `docker save` puis chargées côté hôte avec `docker load` avant `skopeo copy`) :

```

for svc in analysis-api result-frontend; do
  skopeo copy --dest-tls-verify=false \
    docker-daemon:localhost:5000/$svc:1.0 \
    docker://127.0.0.1:5000/$svc:1.0
done

curl -s http://127.0.0.1:5000/v2/_catalog
# -> {"repositories":["analysis-api","result-frontend","sample-api"]}

```

```

mikaël@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ curl -s http://127.0.0.1:5000/v2/_catalog
curl -s http://127.0.0.1:5000/v2/sample-api/tags/list
curl -s http://127.0.0.1:5000/v2/analysis-api/tags/list
curl -s http://127.0.0.1:5000/v2/result-frontend/tags/list
Handling connection for 5000
{"repositories":["analysis-api","result-frontend","sample-api"]}
Handling connection for 5000
{"name":"sample-api","tags":["1.0"]}
Handling connection for 5000
{"name":"analysis-api","tags":["1.0"]}
Handling connection for 5000
{"name":"result-frontend","tags":["1.0"]}

```

Catalog final 3 services

#### 4.4.4 Preuve que le cluster pull bien depuis la registry

Pour démontrer formellement que les pods récupèrent l'image depuis la registry (et non depuis un cache local), on bascule `sample-api` `imagePullPolicy: Always`, on supprime l'image cachée dans Minikube, puis on relance le rollout :

```

kubectl -n labotrack patch deploy/sample-api --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/imagePullPolicy","value":"Always"}]'
minikube ssh -- "docker rmi localhost:5000/sample-api:1.0"
kubectl -n labotrack rollout restart deploy/sample-api
kubectl -n labotrack rollout status deploy/sample-api --timeout=180s
kubectl -n labotrack describe pod -l app=sample-api | sed -n '/Events:/, $p'

```

```

#mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes/labotrack$ kubectl -n labotrack get pods -o wide -l app=sample-api
POD=$(kubectl -n labotrack get pod -l app=sample-api -o jsonpath='{.items[0].metadata.name}')
echo "=== Events du pod $POD ==="
kubectl -n labotrack describe pod $POD | sed -n '/Events:/,$p'
NAME                 READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
sample-api-66bddd864-2kvk4    2/2     Running   0          3m33s   10.244.0.43   minikube   <none>           <none>
sample-api-66bddd864-h5hn6    2/2     Running   0          3m52s   10.244.0.42   minikube   <none>           <none>
=== Events du pod sample-api-66bddd864-2kvk4 ===
Events:
  Type     Reason      Age    From          Message
  ----     ------      -
Normal    Scheduled   3m34s  default-scheduler  Successfully assigned labotrack/sample-api-66bddd864-2kvk4 to minikube
Normal    Pulled      3m33s  kubelet        spec.initContainers(linkerd-init): Container image "cr.l5d.io/linkerd/proxy:edge-26.5.1" already present on machine and can be accessed by the pod
Normal    Created     3m33s  kubelet        spec.initContainers(linkerd-init): Container created
Normal    Started     3m33s  kubelet        spec.initContainers(linkerd-init): Container started
Normal    Pulled      3m33s  kubelet        spec.containers(linkerd-proxy): Container image "cr.l5d.io/linkerd/proxy:edge-26.5.1" already present on machine and can be accessed by the pod
Normal    Created     3m33s  kubelet        spec.containers(linkerd-proxy): Container created
Normal    Started     3m32s  kubelet        spec.containers(linkerd-proxy): Container started
Normal    Pulling     3m31s  kubelet        spec.containers(sample-api): Pulling image "localhost:5000/sample-api:1.0"
Normal    Pulled      3m31s  kubelet        spec.containers(sample-api): Successfully pulled image "localhost:5000/sample-api:1.0" in 23ms (23ms including waiting). Image size: 340034846 bytes.
Normal    Created     3m31s  kubelet        spec.containers(sample-api): Container created
Normal    Started     3m31s  kubelet        spec.containers(sample-api): Container started

```

Pull depuis la registry — events kubelet

L'événement clef dans la sortie est :

```

Normal    Pulling     ...    Pulling image "localhost:5000/sample-api:1.0"
Normal    Pulled      ...    Successfully pulled image "localhost:5000/sample-api:1.0" in 23ms ... Image size: 340034846 bytes.

```

→ Le kubelet a explicitement contacté la registry et téléchargé l'image (340 Mo). Les images Linkerd (linkerd-init, linkerd-proxy) se quant à elles « already present on machine » car elles ont été cachées lors de l'installation initiale du mesh.

## 4.5 Manifests Kubernetes

| Fichier                        | Rôle                                                                       |
|--------------------------------|----------------------------------------------------------------------------|
| 00-namespace.yaml              | Namespace labotrack annoté linkerd.io/inject=enabled                       |
| 10-postgres.yaml               | StatefulSet Postgres + Service headless + Secret + ConfigMap               |
| 20-sample-api.yaml             | Deployment 2 replicas + Service ClusterIP, profil Spring prod              |
| 30-analysis-api.yaml           | Deployment 3 replicas + Service ClusterIP, latence simulée 300             |
| 40-result-frontend.yaml        | Deployment 1 replica + Service NodePort 30080                              |
| 60-linkerd-serviceprofile.yaml | ServiceProfile (retries idempotents, timeouts)                             |
| 70-linkerd-authz.yaml          | Server + MeshTLSAuthentication + AuthorizationPolicy zero-trus             |
| 99-rogue-pod.yaml              | Pod « pirate » optionnel pour démontrer le rejet par l'AuthorizationPolicy |

État du cluster après déploiement complet :

```

mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ kubectl -n labotrack get pods -o wide
NAME                 READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
analysis-api-75f6684b8d-bwtmr    2/2     Running   0          176m   10.244.0.16   minikube   <none>           <none>
analysis-api-75f6684b8d-fzvm2    2/2     Running   0          176m   10.244.0.14   minikube   <none>           <none>
analysis-api-75f6684b8d-tx5kr    2/2     Running   0          176m   10.244.0.15   minikube   <none>           <none>
postgres-0                2/2     Running   0          176m   10.244.0.11   minikube   <none>           <none>
result-frontend-75c7b9b6c6-d8rnr    2/2     Running   0          159m   10.244.0.29   minikube   <none>           <none>
sample-api-c48494c7-fb784        2/2     Running   0          176m   10.244.0.12   minikube   <none>           <none>
sample-api-c48494c7-x74kd        2/2     Running   0          176m   10.244.0.13   minikube   <none>           <none>

```

Pods 2/2 Running (app + linkerd-proxy)

```

mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ kubectl -n labotrack get svc
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
analysis-api ClusterIP    10.107.47.164 <none>         8080/TCP         177m
postgres     ClusterIP    None           <none>         5432/TCP         177m
result-frontend NodePort    10.103.195.128 <none>         8080:30080/TCP   177m
sample-api   ClusterIP    10.102.54.200 <none>         8080/TCP         177m

```

Services ClusterIP + NodePort 30080

Preuve concrète de l'injection Linkerd dans un pod :

```

mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ POD=$(kubectl -n labotrack get pod -l app=sample-api -o jsonpath='{.items[0].metadata.name}')
echo "pod=$POD"
kubectl -n labotrack get pod $POD -o jsonpath='{.spec.containers[*].name}'
echo
pod=sample-api-c48494c7-fb784
linkerd-proxy sample-api

```

Sidecar linkerd-proxy injecté dans sample-api

Chaque pod expose deux containers : le container applicatif **et** linkerd-proxy .

## 4.6 Service Mesh Linkerd

Installation :

```

kubectl apply --server-side -f \
  https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.1/standard-install.yaml
linkerd install --crds | kubectl apply -f -
linkerd install --set proxyInit.runAsRoot=true | kubectl apply -f -

```



```
linkerd check
linkerd viz install | kubectl apply -f -
linkerd viz check
```

Vérifications de santé :

```
linkerd-viz
-----
✓ linkerd-viz Namespace exists
✓ can initialize the client
✓ linkerd-viz ClusterRoles exist
✓ linkerd-viz ClusterRoleBindings exist
✓ tap API server has valid cert
✓ tap API server cert is valid for at least 60 days
✓ tap API service is running
✓ linkerd-viz pods are injected
✓ viz extension pods are running
✓ viz extension proxies are healthy
✓ viz extension proxies are up-to-date
✓ viz extension proxies and cli versions match
✓ prometheus is installed and configured correctly
✓ viz extension self-check

Status check results are ✓
```

linkerd check

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ linkerd viz check
linkerd-viz
-----
✓ linkerd-viz Namespace exists
✓ can initialize the client
✓ linkerd-viz ClusterRoles exist
✓ linkerd-viz ClusterRoleBindings exist
✓ tap API server has valid cert
✓ tap API server cert is valid for at least 60 days
✓ tap API service is running
✓ linkerd-viz pods are injected
✓ viz extension pods are running
✓ viz extension proxies are healthy
✓ viz extension proxies are up-to-date
✓ viz extension proxies and cli versions match
✓ prometheus is installed and configured correctly
✓ viz extension self-check

Status check results are ✓
```

linkerd viz check

### mTLS automatique (objectif énoncé)

```
linkerd -n labotrack viz edges deploy
```

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ linkerd -n labotrack viz edges deploy
SRC      DST      SRC_NS    DST_NS    SECURED
analysis-api  sample-api  labotrack  labotrack  ✓
result-frontend  analysis-api  labotrack  labotrack  ✓
result-frontend  sample-api  labotrack  labotrack  ✓
prometheus  analysis-api  linkerd-viz  labotrack  ✓
prometheus  result-frontend  linkerd-viz  labotrack  ✓
prometheus  sample-api  linkerd-viz  labotrack  ✓
```

mTLS edges — toutes les arêtes SECURED

Toutes les communications inter-pods (result-frontend → sample-api, result-frontend → analysis-api, analysis-api → sample-api, celles depuis Prometheus) sont **chiffrées et authentifiées par certificats** générés automatiquement par Linkerd. Aucune configuration applicative requise.

### Monitoring (stat / tap / top — objectif énoncé)

```
linkerd -n labotrack viz stat deploy
```

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ linkerd -n labotrack viz stat deploy
NAME      MESHED  SUCCESS  RPS    LATENCY_P50  LATENCY_P95  LATENCY_P99  TCP_CONN
analysis-api  3/3    100.00%  1.7rps  1ms          1ms          1ms          9
result-frontend  1/1    100.00%  0.6rps  1ms          1ms          1ms          2
sample-api    2/2    100.00%  1.1rps  1ms          1ms          1ms          12
```

viz stat — RPS, latences, 100 % succès

```
linkerd -n labotrack viz top deploy/analysis-api
```

```
(press q to quit)
(press a/LeftArrowKey to scroll left, d/RightArrowKey to scroll right)

Source      Destination      Method  Path                                     Count  Best  Worst  Last  Success Rate
10.244.0.1  analysis-api-75f6684b8d-fzvm2  GET     /actuator/health/readiness             3      1ms   2ms    1ms   100.00%
10.244.0.1  analysis-api-75f6684b8d-bwtmr  GET     /actuator/health/readiness             3      1ms   2ms    1ms   100.00%
10.244.0.1  analysis-api-75f6684b8d-tx5kr  GET     /actuator/health/readiness             3      1ms   1ms    1ms   100.00%
10.244.0.1  analysis-api-75f6684b8d-fzvm2  GET     /actuator/health/liveness              1      1ms   1ms    1ms   100.00%
10.244.0.1  analysis-api-75f6684b8d-tx5kr  GET     /actuator/health/liveness              1      1ms   1ms    1ms   100.00%
10.244.0.1  analysis-api-75f6684b8d-bwtmr  GET     /actuator/health/liveness              1      1ms   1ms    1ms   100.00%
```

viz top — routes les plus appelées

## 4.7 ServiceProfile — retries & timeouts

Pour `sample-api`, les `GET /samples` et `GET /samples/{id}` sont marqués `isRetryable: true` (idempotents) avec un timeout de 3 s. Le `POST /samples` ne sont pas retryables (effet de bord). Un `retryBudget` (20 % + 10 RPS minimum, TTL 10 s) borne le nombre de retries | éviter les amplifications de panne.

Pour `analysis-api`, le `POST /analyze/{id}` a un timeout de 8 s (compatible avec la latence simulée de 300 ms), tandis que les `GET /results` sont retryables avec timeout de 3 s.

Les routes définies dans le `ServiceProfile` apparaissent dans `linkerd viz top` (ci-dessus) et dans le dashboard graphique (cf. § 4.10).

## 4.8 ServerAuthorization — zero-trust

Chaque service expose un objet `Server` Linkerd. Les autorisations sont :

| Backend                      | Callers autorisés                                                            |
|------------------------------|------------------------------------------------------------------------------|
| <code>sample-api</code>      | <code>result-frontend</code> + <code>analysis-api</code> (identités MeshTLS) |
| <code>analysis-api</code>    | <code>result-frontend</code> (identité MeshTLS)                              |
| <code>result-frontend</code> | tout le monde (NetworkAuthentication 0.0.0.0/0, exposé en NodePort)          |

Tout autre appelant en intra-cluster (ex. un pod pirate dans un autre namespace) reçoit `HTTP 403`. Démonstration via `kubectl apply -f labotrack/manifests/99-rogue-pod.yaml` puis `curl` interne :

```
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ kubectl apply -f labotrack/manifests/99-rogue-pod.yaml
namespace/rogue created
pod/rogue created
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ kubectl -n rogue wait --for=condition=Ready pod/rogue --timeout=60s
pod/rogue condition met
mikael@DESKTOP-LN3JADJ:/mnt/c/Users/mikae/Documents/Dev/INSA/Architecture/tp-kubernetes$ kubectl -n rogue exec rogue -- curl -v --max-time 6 \
http://sample-api.labotrack.svc.cluster.local:8080/samples 2>&1 | tail -20
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0      0      0 --:--:-- --:--:-- --:--:--    0* Host sample-api.labotrack.svc.cluster.local:8080 was resolved.
* IPv6: (none)
* IPv4: 10.102.54.200
* Trying 10.102.54.200:8080...
* Connected to sample-api.labotrack.svc.cluster.local (10.102.54.200) port 8080
* using HTTP/1.x
> GET /samples HTTP/1.1
> Host: sample-api.labotrack.svc.cluster.local:8080
> User-Agent: curl/8.10.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< content-length: 0
< date: Sat, 02 May 2026 19:53:21 GMT
<
  0     0     0     0     0     0      0      0 --:--:-- --:--:-- --:--:--    0
* Connection #0 to host sample-api.labotrack.svc.cluster.local left intact
```

Zero-trust : pod pirate -> HTTP 403

La trace verbose `curl -v` confirme la fermeture par le proxy Linkerd avant même que la requête atteigne l'application — c'est l'application stricte de la politique zero-trust.

## 4.9 RunBook

Le script idempotent `labotrack/runbook.sh` enchaîne en une commande :

1. `minikube start` (si nécessaire).
2. Installation Gateway API CRDs + Linkerd CRDs + control plane + Viz.
3. Build des 3 images en stratégie `registry` (par défaut) ou `in-cluster`.
4. `kubectl apply -f labotrack/manifests/.`
5. Attente des rollouts.
6. Affichage de l'URL du frontend.

## 4.10 Déroulement de la démo



Frontend — UI complète

État initial (registry vide) :

## LaboTrack — Mini-LIS Sample Tracker

3 microservices · result-frontend → (sample-api, analysis-api) · meshed with Linkerd.

### Register a sample

Patient

Exam type

glycemia

Sample type

blood

Register

### Samples & results

| ID                                   | Patient | Exam | Status | Result | Interpretation | Action |
|--------------------------------------|---------|------|--------|--------|----------------|--------|
| No samples yet — register one above. |         |      |        |        |                |        |

Frontend vierge

Après enregistrement de 4 patients et déclenchement de leurs analyses :

## LaboTrack — Mini-LIS Sample Tracker

3 microservices · result-frontend → (sample-api, analysis-api) · meshed with Linkerd.

### Register a sample

Patient

Exam type

glycemia

Sample type

blood

Register

### Samples & results

| ID | Patient | Exam     | Status    | Result   | Interpretation | Action |
|----|---------|----------|-----------|----------|----------------|--------|
| 1  | Dupont  | glycemia | VALIDATED | 1.05 g/L | normal         |        |
| 2  | Martin  | glycemia | VALIDATED | 0.69 g/L | low            |        |
| 3  | Curie   | glycemia | VALIDATED | 0.78 g/L | normal         |        |
| 4  | Pasteur | glycemia | VALIDATED | 0.71 g/L | low            |        |

Frontend avec 4 échantillons VALIDATED

Chaque ligne montre la chaîne complète :

- Création** via `POST /samples` (statut `REGISTERED`).
- Analyse** via `POST /analyze/{id}` qui appelle en interne `sample-api` (preuve d'inter-service).
- Validation** : `analysis-api` met le statut à `VALIDATED` et la valeur de glycémie + son interprétation ( `low` / `normal` / `high` )

apparaissent dans le tableau.

Le smoke test automatique correspondant est sauvegardé dans `labotrack/docs/smoke-test.log`.

Linkerd Viz — UI graphique

Vue d'ensemble des namespaces :

| LINKERD Namespaces                                       |        |              |                  |                   |             |             |
|----------------------------------------------------------|--------|--------------|------------------|-------------------|-------------|-------------|
| CLUSTER                                                  |        |              |                  |                   |             |             |
| Namespaces                                               |        |              |                  |                   |             |             |
| Control Plane                                            |        |              |                  |                   |             |             |
| DEFAULT                                                  |        |              |                  |                   |             |             |
| WORKLOADS                                                |        |              |                  |                   |             |             |
| Cron Jobs                                                |        |              |                  |                   |             |             |
| Daemon Sets                                              |        |              |                  |                   |             |             |
| Deployments                                              |        |              |                  |                   |             |             |
| Services                                                 |        |              |                  |                   |             |             |
| Jobs                                                     |        |              |                  |                   |             |             |
| Pods                                                     |        |              |                  |                   |             |             |
| Replica Sets                                             |        |              |                  |                   |             |             |
| Replication Controllers                                  |        |              |                  |                   |             |             |
| Stateful Sets                                            |        |              |                  |                   |             |             |
| TOOLS                                                    |        |              |                  |                   |             |             |
| Tap                                                      |        |              |                  |                   |             |             |
| Top                                                      |        |              |                  |                   |             |             |
| Routes                                                   |        |              |                  |                   |             |             |
| Community                                                |        |              |                  |                   |             |             |
| Extensions                                               |        |              |                  |                   |             |             |
| Documentation                                            |        |              |                  |                   |             |             |
| GitHub                                                   |        |              |                  |                   |             |             |
| Mailing List                                             |        |              |                  |                   |             |             |
| Slack                                                    |        |              |                  |                   |             |             |
| Running Linkerd 26.5.1 (edge).<br>Linkerd is up to date. |        |              |                  |                   |             |             |
| HTTP Metrics                                             |        |              |                  |                   |             |             |
| Namespace                                                | Meshed | Success Rate | RPS              | P50 Latency       | P95 Latency | P99 Latency |
| default                                                  | 0/0    | —            | —                | —                 | —           | —           |
| kube-node-lease                                          | 0/0    | —            | —                | —                 | —           | —           |
| kube-public                                              | 0/0    | —            | —                | —                 | —           | —           |
| kube-system                                              | 0/8    | —            | —                | —                 | —           | —           |
| kubernetes-dashboard                                     | 0/2    | —            | —                | —                 | —           | —           |
| labotrack                                                | 7/7    | 100.00%      | 3.67             | 1 ms              | 1 ms        | 1 ms        |
| linkerd                                                  | 3/3    | 100.00%      | 2.4              | 1 ms              | 1 ms        | 2 ms        |
| linkerd-viz                                              | 5/5    | 100.00%      | 5.82             | 1 ms              | 9 ms        | 13 ms       |
| TCP Metrics                                              |        |              |                  |                   |             |             |
| Namespace                                                | Meshed | Connections  | Read Bytes / sec | Write Bytes / sec |             |             |
| default                                                  | 0/0    | —            | —                | —                 |             |             |
| kube-node-lease                                          | 0/0    | —            | —                | —                 |             |             |
| kube-public                                              | 0/0    | —            | —                | —                 |             |             |
| kube-system                                              | 0/8    | —            | —                | —                 |             |             |
| kubernetes-dashboard                                     | 0/2    | —            | —                | —                 |             |             |
| labotrack                                                | 7/7    | 23           | 409.38/s         | 11.27KB/s         |             |             |
| linkerd                                                  | 3/3    | 45           | 269.27B/s        | 9.057KB/s         |             |             |
| linkerd-viz                                              | 5/5    | 28           | 748.38B/s        | 14.402KB/s        |             |             |

Linkerd Viz — vue Namespaces

Topologie auto-découverte du namespace LaboTrack (les flèches sont les flux REST réels capturés par les sidecars) :

| Namespace > labotrack |        |              |      |             |             |             |
|-----------------------|--------|--------------|------|-------------|-------------|-------------|
|                       |        |              |      |             |             |             |
| Deployments           |        |              |      |             |             |             |
| Deployment            | Meshed | Success Rate | RPS  | P50 Latency | P95 Latency | P99 Latency |
| analysis-api          | 3/3    | 100.00%      | 1.7  | 1 ms        | 1 ms        | 1 ms        |
| result-frontend       | 1/1    | 100.00%      | 0.57 | 1 ms        | 1 ms        | 1 ms        |
| sample-api            | 2/2    | 100.00%      | 1.13 | 1 ms        | 1 ms        | 1 ms        |

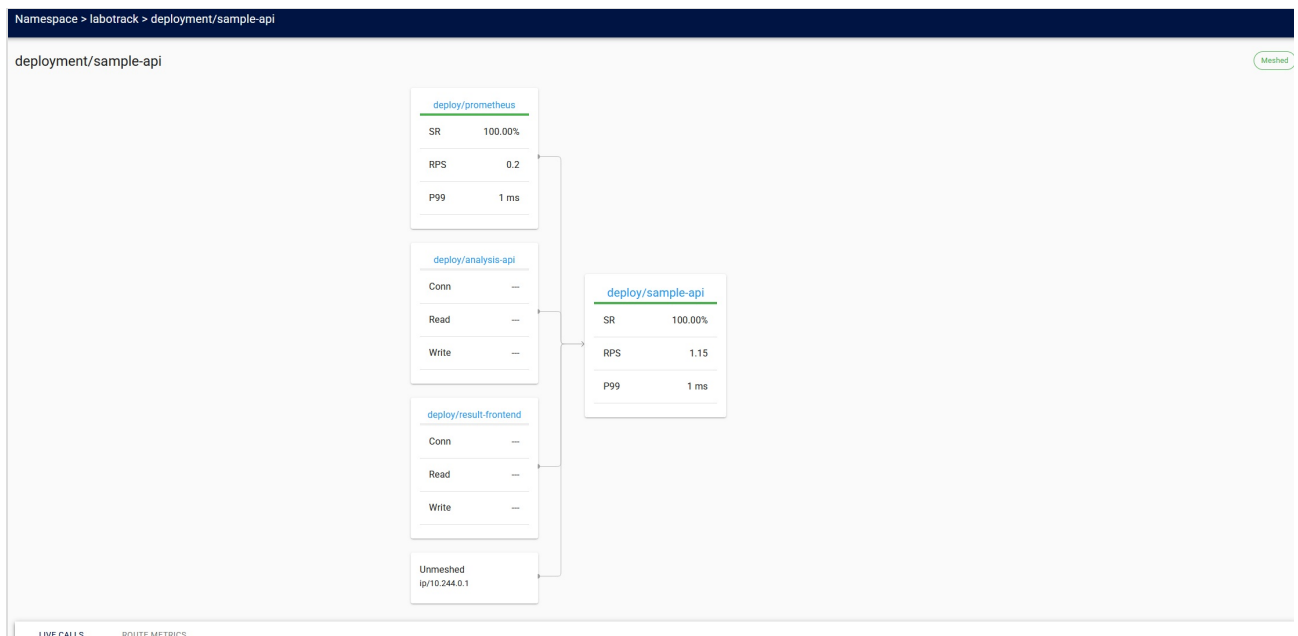
Linkerd Viz — topologie LaboTrack

Métriques HTTP + TCP par déploiement (1-2 ms p99, 100 % de succès, 100 % maillé) :

| Namespace > labotrack > Deployments |        |              |                  |                   |             |             |
|-------------------------------------|--------|--------------|------------------|-------------------|-------------|-------------|
| HTTP Metrics                        |        |              |                  |                   |             |             |
| Deployment                          | Meshed | Success Rate | RPS              | P50 Latency       | P95 Latency | P99 Latency |
| analysis-api                        | 3/3    | 100.00%      | 1.72             | 1 ms              | 1 ms        | 1 ms        |
| result-frontend                     | 1/1    | 100.00%      | 0.58             | 1 ms              | 2 ms        | 2 ms        |
| sample-api                          | 2/2    | 100.00%      | 1.15             | 1 ms              | 1 ms        | 1 ms        |
| TCP Metrics                         |        |              |                  |                   |             |             |
| Deployment                          | Meshed | Connections  | Read Bytes / sec | Write Bytes / sec |             |             |
| analysis-api                        | 3/3    | 9            | 189.33B/s        | 5.352KB/s         |             |             |
| result-frontend                     | 1/1    | 2            | 60.35B/s         | 2.078KB/s         |             |             |
| sample-api                          | 2/2    | 12           | 145.77B/s        | 2.978KB/s         |             |             |

Linkerd Viz — métriques par déploiement

Vue détaillée d'un service ( `sample-api` ) avec ses pods et routes :



## 5. Limites et pistes d'amélioration

- **Postgres** en single-replica (acceptable pour la démo, pas pour la production — opérateur recommandé en prod).
- **Schéma JPA** géré par `ddl-auto=update` (Flyway/Liquibase recommandé en prod).
- **Pas d'Ingress** : le frontend est exposé en NodePort 30080. Une `Ingress` + `cert-manager` serait préférable hors Minikube.
- **Prometheus/Grafana standalone** non livré : Linkerd Viz embarque déjà les deux. La mise en place d'un kube-prometheus-stack indépendant est laissée comme amélioration.
- **Daemon Docker WSL2 + insecure-registries** : la directive `insecure-registries` n'a pas été honorée par notre instance de da Docker, contournée par `skopeo`. Investigation plus poussée à mener (peut-être un bug spécifique à la version du paquet `docker-c WSL2`).

## 6. Annexes

- `architecture.md` — diagramme et choix de design.
- `manuel-technique.md` — détails de la stack et des manifests.
- `manuel-utilisateur.md` — guide de démarrage et dépannage.
- `runbook.sh` — script idempotent de déploiement.
- `bootstrap-wsl.sh` — script d'amorçage de l'environnement WSL2.
- `smoke-test.log` — trace texte du flow end-to-end.
- `zero-trust.log` — trace texte de la démo zero-trust.